One TrustX

คู่มือโครงการฉบับสมบูรณ์: Baggage Disruption Intelligence & Recovery Coordination

ฉบับปรับตาม insight โดยตรงจาก SKY Innovation Spark 2026 · 7 ตุลาคม 2026

เอกสารนี้อธิบาย One TrustX ตั้งแต่เหตุผลที่สร้างโครงการ การเปลี่ยนทิศทางล่าสุด ปัญหาที่เลือกแก้ ผู้ใช้ คุณค่าทางธุรกิจ ข้อมูลที่ต้องใช้ สิ่งที่ระบบทำและยังทำไม่ได้ วิธีนำเสนอ ไปจนถึงรายละเอียดเชิงเทคนิคในภาคผนวก
โดยช่วงต้นเขียนให้ผู้อ่านที่ไม่ใช่สายเทคเข้าใจได้ และช่วงท้ายใช้เป็นหลักฐานสำหรับทีมพัฒนา ผู้เชี่ยวชาญ และคำถามเชิงลึกจากกรรมการ

ส่วนที่ 1: สรุปสำหรับผู้บริหารและกรรมการ

1.1 One TrustX คืออะไร

One TrustX คือระบบช่วยตัดสินใจสำหรับการรับมือเหตุขัดข้องในการปฏิบัติการสนามบิน โดยเริ่มจากปัญหาเดียวที่ชัดเจนคือ กระเป๋าขาออกล่าช้าและเสี่ยงไปไม่ทันเที่ยวบิน

ระบบไม่ได้เข้ามาแทน Baggage Reconciliation System (BRS), Baggage Handling System (BHS), Airport Operational Database (AODB) หรือ Airport Collaborative Decision Making (A-CDM)
แต่ใช้สัญญาณจากระบบเหล่านั้นร่วมกับข้อมูลกำลังคน อุปกรณ์ เที่ยวบินต่อ และเหตุภายนอก
เพื่อช่วยตอบคำถามสำคัญสี่ข้อในจอเดียว:

1. ตอนนี้กระบวนการกระเป๋าปกติหรือไม่
2. ถ้าไม่ปกติ ปัญหาอยู่ที่ขั้นไหนและน่าจะเกิดจากอะไร
3. ปัญหาจะกระทบเที่ยวบินหรือกระเป๋ากลุ่มใดต่อ
4. เจ้าหน้าที่ควรพิจารณาแก้อะไรก่อน ใครเป็นผู้รับผิดชอบ และต้องทำก่อนเมื่อใด

ประโยคสรุปที่ต้องการให้กรรมการจำได้คือ:

BRS tracks and reconciles the bag. One TrustX creates a cross-operational recovery view around the disruption.

หรือภาษาไทย:

BRS ช่วยติดตามและตรวจสอบกระเป๋า ส่วน One TrustX
ช่วยให้ทีมปฏิบัติการเห็นภาพรวมของเหตุขัดข้องและจัดลำดับการกู้สถานการณ์

1.2 ปัญหาที่เลือกแก้

ปัญหาหลักคือ เมื่อกระเป๋าเริ่มค้างที่ขั้นใดขั้นหนึ่ง ข้อมูลที่ใช้เข้าใจสถานการณ์มักอยู่คนละระบบและคนละฝ่าย เช่น:

- BRS/BHS เห็น milestone และสถานะการลำเลียง
- AODB เห็นเวลาเที่ยวบินและ gate
- Ground Handler เห็นจำนวนพนักงาน รถลาก cart หรือ ULD
- สายการบินเห็นเที่ยวบินต่อและเวลาที่เหลือ
- ฝ่ายสนามบินเห็นข้อจำกัดของพื้นที่และระบบ
- แหล่งภายนอกเห็นฝน น้ำท่วม หรือเหตุที่ทำให้คนและอุปกรณ์มาถึงล่าช้า

ข้อมูลแต่ละส่วนอาจถูกต้องในขอบเขตของตนเอง แต่ผู้ควบคุมงานยังต้องนำมาประกอบกันเพื่อรู้ว่าเป็นปัญหาเฉพาะกระเป๋าหนึ่งใบ หรือกำลังกลายเป็น backlog ที่กระทบหลายเที่ยวบิน One TrustX จึงเน้นการสร้าง shared recovery view
ไม่ใช่สร้างระบบติดตามกระเป๋าใหม่

1.3 ใครเป็นผู้ใช้

บทบาท	หน้าที่ในปัญหานี้	สิ่งที่ต้องการจากระบบ
Baggage/Ground Operations Controller	จัดลำดับงาน คน และอุปกรณ์	จุดคอขวด ผลกระทบ และ action ที่ควรพิจารณาก่อน
Ground Handler Supervisor	บริหารทีม รถลาก cart และ ULD	งานที่ต้องเร่งและ deadline
Airline Operations Control (OCC)	ตัดสินใจด้านเที่ยวบินและ connection	กระเป๋าที่เสี่ยงและผลต่อ flight readiness
Airport Operations	ประสานทรัพยากรและระบบสนามบิน	ภาพรวมข้ามฝ่ายและสถานะการแก้ไข
ผู้โดยสาร	ผู้ได้รับผลกระทบปลายทาง	กระเป๋ามาถึงถูกต้องและได้รับข้อมูลที่น่าเชื่อถือ

ผู้ใช้หลักของหน้าจอในรอบนี้คือ Baggage/Ground Operations Controller ไม่ใช่ผู้โดยสาร
เพราะโจทย์หลักคือการตรวจพบและกู้สถานการณ์ก่อนผลกระทบลามไปถึงผู้โดยสาร

1.4 คุณค่าที่เสนอ

One TrustX เสนอคุณค่า 5 ด้าน:

- Visibility: เห็นสถานะกระบวนการกระเป๋าทีละขั้น ไม่ใช่เฉพาะรายการ exception
- Context: เชื่อมสถานะกระเป๋ากับเส้นตายเที่ยวบิน คน อุปกรณ์ และเหตุภายนอก
- Prioritisation: จัดลำดับว่าควรช่วยเที่ยวบินหรือกลุ่มกระเป๋าใดก่อน
- Coordination: ระบุ owner, dependency และ deadline ของ action ให้แต่ละฝ่ายเห็นตรงกัน
- Accountability: เก็บที่มาข้อมูล เหตุผล คำแนะนำ การอนุมัติ และประวัติการตัดสินใจ

ระบบไม่สั่งการอุปกรณ์สนามบินโดยอัตโนมัติ เจ้าหน้าที่เป็นผู้ Accept, Reject หรือ Override คำแนะนำเสมอ

ส่วนที่ 2: เหตุผลที่โครงการเปลี่ยนทิศทาง

2.1 แนวคิดเดิม

แนวคิดเดิมของ One TrustX คือ Trusted Operational Coordination Layer สำหรับหลายฝ่ายใน aviation โดยพยายามรวมข้อมูล สร้าง Common Operational State ใช้ AI ช่วยวิเคราะห์ และรองรับหลาย use case เช่น passenger flow, flight delay, gate, baggage และ disruption

แนวคิดนี้มีจุดแข็งด้าน architecture แต่กว้างเกินไปสำหรับ pitch 7 นาที เพราะกรรมการอาจไม่เห็นว่า:

- ปัญหาแรกที่จะแก้คืออะไร
- ใครเป็นผู้ใช้หลัก
- สาเหตุของปัญหาคืออะไร
- ต้องขอข้อมูลใดจากใคร
- ผลลัพธ์บนหน้าจอช่วยตัดสินใจอะไร
- แตกต่างจากผลิตภัณฑ์ของ SKY ที่มีอยู่แล้วอย่างไร

2.2 Insight โดยตรงจาก SKY และ mentor

ข้อมูลจาก Kick-off และคำแนะนำของ mentor ระบุประเด็นสำคัญว่า:

- ให้เลือกเพียงหนึ่งปัญหาที่ต้องการแก้จริง
- อธิบายว่าปัญหาเกิดจากขั้นใดและมีเหตุอะไรบ้าง
- ระบุให้ชัดว่าแก้ปัญหาให้ใคร
- ระบุว่าต้องแชร์ข้อมูลส่วนใด จากฝ่ายใด
- หน้าจอต้องมองแล้วรู้ได้ทันทีว่าไม่มีปัญหา หรือปัญหาเกิดที่ส่วนไหน
- เรื่องการขยายไป use case อื่นควรกล่าวเฉพาะช่วงท้าย
- SKY มี Smart Flows, BRS และ AODB อยู่แล้ว และกำลังเดินไปสู่ A-CDM/TAM และ Unified Data Platform

- ในกระบวนการกระเป๋า จุดเสี่ยงสำคัญอยู่ที่ handoff หรือการเปลี่ยนความรับผิดชอบระหว่างฝ่าย
- เป้าหมายสนามบินและ aviation hub ต้องพิจารณา People, Process และ IT ร่วมกัน ไม่ใช่ติดตั้งระบบเพิ่มโดยไม่แก้ workflow

Insight นี้ทำให้ทีมตัดสินใจว่า One TrustX ต้องไม่เล่าว่าเป็นแพลตฟอร์มรวมทุกอย่าง แต่ต้องแสดงให้เห็นว่า architecture เดิมนำมาแก้ปัญหากระเป๋าที่เฉพาะเจาะจงได้อย่างไร

2.3 สิ่งที่เปลี่ยน และสิ่งที่ยังคงเดิม

ด้าน	ก่อนปรับ	หลังปรับล่าสุด
Hero use case	การประสานงานสนามบินหลายเรื่อง	กระเป๋าขาออกล่าช้าและ disruption recovery
Problem statement	ข้อมูลหลายฝ่ายไม่เชื่อมกัน	กระเป๋าค้างที่ขั้นไหน เพราะอะไร กระทบใคร และแก้อะไรก่อน
ผู้ใช้	หลาย stakeholder พร้อมกัน	Baggage/Ground Operations Controller เป็นหลัก
หน้าหลัก	Common Operational State ทั่วไป	Baggage Delay Control View
Positioning	coordination platform กว้าง	focused recovery-prioritisation layer
ระบบเดิม	event, trust, simulation, recommendation, audit	คงไว้ทั้งหมดและนำมาใช้กับ baggage
Future scope	ปรากฏตลอดเรื่อง	กล่าวช่วงท้าย: turnaround, gate, passenger, cargo

การเปลี่ยนนี้เป็นการเปลี่ยน จุดเริ่มต้นและเรื่องเล่า ไม่ใช่การรื้อ architecture เดิม

2.4 ลำดับวิวัฒนาการของโครงการจนถึงเวอร์ชันปัจจุบัน

ช่วง	แนวคิด/สิ่งที่พัฒนา	ข้อมูลหรือเหตุผลที่ทำให้เปลี่ยน	ผลต่อเวอร์ชันปัจจุบัน
ระยะเริ่มต้น	โมดูล queue simulation สำหรับคาดการณ์คิว ทดลองเปิดเลน/redirect และ Passenger Safe Time	ต้องการสาธิต decision support ด้วยข้อมูลที่หาได้และจำลองซ้ำได้	ได้ simulation engine, what-if, human decision, audit และ passenger modules ซึ่งยังคงอยู่
ระยะ One TrustX แรก	เพิ่ม Trusted Event Layer, Common Operational State และการประสานหลาย stakeholder	ปัญหาการตัดสินใจไม่ได้มีเพียงคิว แต่เกิดจากข้อมูลหลายแหล่งไม่ตรงกัน	ได้ event lineage, conflict reconciliation, recommendation และ scenario validation
ระยะขยาย scenario	เพิ่ม operational/disaster scenarios และ failure injection	ต้องการทดสอบ missing, late, duplicate, contradictory และ stale events	ได้ scenario library และ validation pack แต่ผลยังเป็น synthetic proxy
หลัง SKY Kick-off	พบว่า SKY มี Smart Flows, BRS, AODB และทิศทาง A-CDM/TAM อยู่แล้ว	แนวคิด “รวมข้อมูลทุกฝ่าย” เสี่ยงซ้ำกับผลิตภัณฑ์/ทิศทางของ SKY	หยุดวาง positioning ว่าเป็นฐานข้อมูลกลางหรือ A-CDM ใหม่
หลัง feedback mentor	ต้องเลือกหนึ่งปัญหา บอกสาเหตุ ผู้ใช้ แหล่งข้อมูล และทำจอที่เห็นปัญหาทันที	Insight โดยตรงจาก SKY Innovation Spark วันที่ 7 ต.ค. 2026	เลือก delayed outbound baggage เป็น hero use case
Scope narrowing	เปลี่ยนจาก baggage tracking เป็น baggage-disruption recovery	BRS ของ SKY ติดตาม milestone เชื่อม AODB/BHS และมี alert บางประเภทอยู่แล้ว	One TrustX เน้น cross-system bottleneck, impact และ recovery priority
เวอร์ชันปัจจุบัน	Baggage Delay Control View + ระบบเดิมทั้งหมด	ต้องแสดง “ปกติไหม–ติดตรงไหน–กระทบอะไร–ทำอะไรต่อ”	เพิ่ม baggage_control contract, stage view, owner/deadline และ unknown-data handling

การเปลี่ยนแต่ละระยะไม่ใช่การลบงานก่อนหน้า แต่เป็นการจัดชั้นใหม่:

```
Queue-simulation capabilities
  queue simulation / passenger journey / what-if
            ↓ เป็น capability สนับสนุน
One TrustX foundation
  trusted events / shared state / conflict / recommendation / audit
            ↓ นำมาใช้กับปัญหาที่เฉพาะขึ้น
Current hero use case
  baggage disruption detection and recovery prioritisation
```

2.5 เหตุผลที่เวอร์ชันปัจจุบันเหมาะกับรอบ Preliminary Pitch

- มีปัญหาเดียวที่อธิบายได้ภายในหนึ่งประโยค
- มีผู้ใช้หลักและ decision ที่ชัดเจน

- เชื่อมกับ pain point เรื่อง handoff ที่ SKY ให้ข้อมูลโดยตรง
- ไม่แข่งขันกับ BRS/AODB แต่เพิ่ม value จากการใช้ข้อมูลร่วมกัน
- แสดง feasibility เพราะเริ่มจาก infrastructure ที่มีอยู่
- มีหน้าจอที่ตอบ feedback mentor โดยตรง
- ยังรักษา vision ระยะยาวของ One TrustX ไว้ แต่ย้ายไปกล่าวในช่วงท้าย

ส่วนที่ 3: กระบวนการกระเป๋าและจุดที่ระบบเข้าไปช่วย

3.1 ภาพกระบวนการแบบง่าย

Passenger handover / Check-in
↓
Screening and Baggage Handling
↓
Sorting
↓
Loading onto aircraft
↓
Transfer / Connection (ถ้ามี)
↓
Unloading and Arrival
↓
Return to passenger

ทุกลูกศรเป็น handoff ซึ่งอาจเกี่ยวข้องกับคน ระบบ อุปกรณ์ และความรับผิดชอบที่ต่างกัน IATA Resolution 753 กำหนดการติดตามอย่างน้อยที่ acceptance, loading, transfer และ arrival เพื่อให้ตรวจสอบ custody ของกระเป๋าได้

3.2 ตัวอย่างสาเหตุความล่าช้า

ขั้น	สาเหตุที่เป็นไปได้	ข้อมูลที่ใช้ตรวจ
Check-in/Acceptance	ผู้โดยสารมากระจุกตัว, bag drop ช้า	transaction timestamp, passenger-flow aggregate
Screening	ส่งตรวจช้า, เครื่องตรวจหรือสายพานขัดข้อง	BHS status, screening exception
Sorting	routing ผิด, chute หรือ conveyor มีปัญหา	sortation event, location, device status
Loading	คนไม่พอ, cart/tug/ULD ไม่พร้อม, gate เปลี่ยน	BRS milestone, resource/task state, AODB
Transfer	ขาเข้าล่าช้า, connection window สั้น	inbound ETA, onward ETD, transfer event
Arrival	unloading หรือ belt allocation ช้า	unload event, belt status, staffing

ตารางนี้เป็นกรอบสมมติฐานสำหรับ validation กับผู้เชี่ยวชาญ ไม่ใช่ข้อสรุปว่าทุกสนามบินมีสาเหตุเหมือนกัน

3.3 ตัวอย่างเหตุการณ์ในระบบจำลอง

ใน controlled fixture ปัจจุบัน:

- เที่ยวบินจำลอง TT401 กำลังเข้าใกล้ boarding close
- Boarding ถูกแจ้งว่า complete
- Gate ถูกแจ้งว่า ready
- แต่ baggage loading ยังอยู่ที่ 82%
- ระบบจึงไม่อนุญาตให้สถานะอื่นกลบ dependency ของกระเป๋า
- Common Operational State เป็น AT_RISK
- Bottleneck คือ Loading
- Recommendation คือให้ Ground Handler พิจารณาส่ง resource สำรอง GH-03 ก่อน boarding close

นี่เป็นตัวอย่างเพื่อพิสูจน์ workflow ของระบบ ไม่ใช่เหตุการณ์จริงของเที่ยวบินหรือสนามบินใด

ส่วนที่ 4: หน้าจอและประสบการณ์ผู้ใช้

4.1 Baggage Delay Control View

หน้าหลักใหม่ออกแบบให้ตอบคำถามตามลำดับความคิดของเจ้าหน้าที่:

NORMAL? → WHERE? → WHY? → IMPACT? → NEXT ACTION? → OWNER? → DEADLINE?

องค์ประกอบหลักประกอบด้วย:

1. Overall baggage-flow status
2. Stage status: Check-in, Screening, Sorting, Loading, Transfer, Arrival
3. Root bottleneck และเหตุผล
4. หลักฐานหรือ source ที่รองรับข้อสรุป
5. เที่ยวบินและ operation ที่ได้รับผล
6. Recovery action ที่แนะนำ
7. Action owner และ deadline
8. Human approval status
9. Data-source boundary และ privacy note

สถานะตัวอย่าง:

```
Check-in   Screening   Sorting   Loading   Transfer   Arrival
  OK         OK         OK       DELAY      RISK       OK
                                   │
                                   ├─ เหตุ: loading milestone ต่ำกว่าแผน
                                   ├─ กระทบ: flight readiness
                                   └─ ทำต่อ: พิจารณาส่ง resource สำรอง
```

4.2 โมดูลเดิมที่ยังอยู่

One TrustX ไม่ได้ตัดความสามารถเดิมออก ระบบยังมี:

- Event Explorer: ดูเหตุการณ์ ที่มา เวลา lineage และ verification
- Conflict Center: ดูข้อมูลที่ขัดแย้งและ reconcile ตามกฎ
- Shared State: ดูสถานะร่วมและหลักฐานประกอบ
- Intelligence: detect, predict และ recommend
- Coordination: owner, dependency และ action status
- Scenario Matrix: ทดลองสถานการณ์หลายระดับ
- Performance: แยกผล prototype ออกจาก production claim
- Audit: บันทึกว่าใครตัดสินใจอะไรและเพราะเหตุใด
- Sources: แสดง provenance และข้อจำกัดของข้อมูล
- โมดูล queue simulation เดิม: queue simulation, what-if, passenger journey และ Safe Time ซึ่งเป็น capability เดิมและไม่ใช่ hero use case ของ pitch รอบนี้

ส่วนที่ 5: การวางตำแหน่งผลิตภัณฑ์

5.1 ตำแหน่งเมื่อเทียบกับระบบของ SKY

ระบบ	บทบาทหลัก	One TrustX ใช้อย่างไร	สิ่งที่ One TrustX เน้นเพิ่ม
BRS	ติดตามและ reconcile กระเป๋าในจุดสำคัญ	รับ milestone/exception เป็น input	มอง disruption ระดับ operation และลำดับ recovery
BHS	ลำเลียง คัดแยก และสถานะอุปกรณ์	รับ processing/device state	เชื่อมอาการของระบบกับผลต่อ flight deadline
AODB	แหล่งข้อมูลกลางด้านเที่ยวบิน	ใช้ ETA/ETD, gate, flight status	ใช้เป็น deadline และ impact context
A-CDM/TAM	collaborative decision ระดับสนามบิน	อาจเชื่อม milestone ในอนาคต	เจาะ use case กระเป๋า ไม่สร้าง A-CDM ใหม่
Smart Flows	การไหลและ dwell time ของผู้โดยสาร	future leading signal ที่ check-in	ยังไม่ใช่ input จำเป็นของ prototype แรก

คำอธิบายที่ปลอดภัยและตรงไปตรงมา:

One TrustX builds on existing BRS/AODB signals and focuses on cross-stakeholder baggage-disruption recovery and prioritisation.

ไม่ควรกล่าวว่า BRS ไม่มี alert หรือไม่สามารถวิเคราะห์ได้ เพราะข้อมูลสาธารณะของ SKY ระบุว่า BRS มี alert สำหรับความผิดปกติบางประเภทอยู่แล้ว ความแตกต่างที่ต้องพิสูจน์คือการเชื่อม baggage + flight + ground resource + connection + external disruption ให้เป็น recovery view ที่เจ้าหน้าที่ใช้ร่วมกัน

5.2 Business positioning

One TrustX ไม่ขายคำว่า “AI ที่รู้ทุกอย่าง” แต่เสนอ:

- Integration-first: ใช้ระบบเดิมให้คุ้มค่า
- Exception-first: โฟกัสช่วงผิดปกติ ไม่สร้าง dashboard ทุกอย่าง
- Human-centred: แนะนำและอธิบาย แต่ผู้ปฏิบัติงานตัดสินใจ
- Evidence-based: ทุกข้อสรุปย้อนกลับไปหา event/source ได้
- Expandable: เริ่มจาก baggage แล้วค่อยขยายตามผล validation

ส่วนที่ 6: ข้อมูลที่ต้องใช้และที่มาของข้อมูล

6.1 ข้อมูลปฏิบัติการที่ต้องขอในอนาคต

กลุ่มข้อมูล	Field ขั้นต่ำ	เจ้าของ/แหล่งที่คาดหวัง	ใช้ตอบคำถาม
Baggage event	pseudonymous bag ID, event, timestamp, station/location, flight, container	BRS/BHS/DCS	ผ่าน milestone หรือยัง ค้างที่ใด
Flight	flight ID, ETA/ETD, gate/stand, status, update time	AODB	deadline และผลกระทบต่อเที่ยวบิน
Ground task	task, owner, planned/actual time, status	Ground Handler	งานใดช้าและใครรับผิดชอบ
Resource	staff availability, cart/tug/ULD availability	Ground Handler	คอขวดเกิดจากคนหรืออุปกรณ์
Equipment	belt/chute/conveyor status, fault code	BHS operator	คอขวดเกิดจากระบบหรือไม่
Connection	inbound/outbound flight, connection cutoff, priority class	Airline	กลุ่ม transfer ใดเร่งด่วน
Disruption	weather, flood, access restriction, incident	แหล่งทางการ	ปัจจัยภายนอกและ early warning

6.2 หลักการลดข้อมูลส่วนบุคคล

Prototype แรกไม่ต้องใช้ชื่อผู้โดยสาร เลขหนังสือเดินทาง หรือรายละเอียดที่ไม่จำเป็นต่อ operations สามารถใช้:

- pseudonymous bag identifier

- aggregate bag count
- stage throughput
- time-to-cutoff
- resource readiness
- connection-risk category

หากต้องเชื่อมข้อมูลที่ระบุตัวบุคคลในอนาคต ต้องมี purpose limitation, access control, retention policy, encryption, audit และ PDPA review

6.3 ข้อมูลที่มีอยู่ใน repository ตอนนี้

ข้อมูลแบ่งเป็น 5 ประเภทเพื่อป้องกันการอ้างเกินจริง:

ประเภท	ความหมาย	ตัวอย่างในโครงการ
REAL_PUBLIC	ข้อมูลจริงจากแหล่งสาธารณะ	Open-Meteo, OurAirports, OpenSky, CAAT, BTS, CBP, UK CAA
SYNTHETIC	ข้อมูลที่ทีมสร้างเพื่อ demo	TT401, baggage events, ground tasks, gates, passengers
ASSUMPTION	ค่าที่ตั้งเพื่อจำลอง	capacity, severity, service rate, resource limit
DERIVED	ค่าที่คำนวณจาก input	shared state, aggregate, risk feature
SIMULATED	ผลการทดลอง engine	predicted delay, recommendation, KPI proxy

ข้อมูลจริงสาธารณะใช้เป็น context และ benchmark เท่านั้น ไม่ถูกเปลี่ยนชื่อให้เป็น operational data ของ BKK

6.4 แหล่งข้อมูลสาธารณะ

แหล่ง	ใช้สำหรับ	ข้อจำกัด
SKY ICT BRS/AODB product pages	เข้าใจ capability สาธารณะและกำหนด positioning	ไม่ยืนยันรายละเอียด implementation ภายใน
IATA Resolution 753	tracking points, custody/handoff framework	เป็นมาตรฐานอุตสาหกรรม ไม่ใช่ข้อมูลเหตุการณ์ BKK
SITA Baggage IT Insights	ขนาดปัญหาและ benchmark อุตสาหกรรม	เป็นข้อมูลภาพรวมโลก ไม่ใช่ KPI ของ SKY
AOT public notice	reference incident เรื่อง delayed baggage	ใช้ยืนยันว่าปัญหามีจริง ไม่ใช่ผลของระบบเรา
Open-Meteo	historical weather context	ไม่ใช่ sensor ภายในสนามบิน
OpenSky	aircraft-state context	ไม่ใช่ manifest, baggage หรือ complete schedule
OurAirports	airport/runway metadata	ไม่ใช่ operational telemetry
CAAT	aggregate aviation statistics ของไทย	coverage ไม่ต่อเนื่องและไม่ใช่ baggage telemetry
BTS/CBP/UK CAA	external benchmark และวิธีประเมิน	คนละประเทศ/กระบวนการ ห้ามใช้แทนข้อมูล BKK

6.5 ตัวเลขอุตสาหกรรมที่ใช้ได้

- SITA Baggage IT Insights 2026 ระบุว่าในปี 2025 มีกระเป๋า mishandled ราว 24 ล้านใบ และต้นทุนอุตสาหกรรมประมาณ 6.3 พันล้านดอลลาร์ต่อปี
- ถ้าใช้ตัวเลขเดียวกับ Kick-off: ปี 2024 คือ 33.4 ล้านใบและประมาณ 5 พันล้านดอลลาร์ ต้องระบุว่าเป็นคนละปี
- ใช้คำว่า mishandled หรือ “จัดการผิดพลาด/ล่าช้า/ส่งผิด/เสียหาย” ไม่ใช้คำว่า “หายทั้งหมด”
- ตัวเลข global ใช้เพื่ออธิบายขนาด pain point ไม่ใช้คำนวณผลตอบแทนของ SKY โดยตรง

ส่วนที่ 7: ระบบทำงานอย่างไรสำหรับคนไม่ใช่สายเทค

7.1 เปรียบเทียบกับห้องควบคุม

ให้นึกว่าแต่ละระบบเป็นพนักงานที่ถือข้อมูลคนละชิ้น:

- BRS บอกว่ากระเป๋าผ่านจุดไหน

- AODB บอกว่าเครื่องออกเมื่อไรและ gate ไหน
- Ground Handler บอกว่ามีคนและรถพร้อมเท่าไร
- BHS บอกว่าสายพานหรือเครื่องคัดแยกทำงานหรือไม่
- Airline บอกว่ากระเป๋าใดมีเที่ยวบินต่อและเหลือเวลาเท่าไร

One TrustX ทำหน้าที่เหมือนโต๊ะกลางที่รวบรวมรายงาน ตรวจว่าข้อมูลสอดคล้องกันหรือไม่ สรุปสถานการณ์ และเสนอรายการที่ควรจัดการก่อน โดยยังให้หัวหน้าปฏิบัติการเป็นผู้ตัดสินใจ

7.2 Journey ตั้งแต่เกิดเหตุถึงปิดงาน

รับ event จากระบบเดิม
↓
ตรวจ source, timestamp, schema, duplicate และ freshness
↓
จัด event เข้ากับเที่ยวบิน กระเป๋า ขั้นตอน และทรัพยากร
↓
สร้างสถานะรายชิ้นและ Common Operational State
↓
ตรวจ exception, conflict และ bottleneck
↓
ประเมิน deadline และ downstream impact
↓
จัดลำดับ recovery action
↓
เจ้าหน้าที่ Accept / Reject / Override
↓
บันทึกเหตุผลและติดตามผลลัพธ์

7.3 ถ้าข้อมูลขัดแย้งหรือเก่า

ระบบไม่ควรซ่อนปัญหา หากแหล่งหนึ่งแจ้ง boarding complete แต่ baggage loading ยังไม่ครบ ระบบต้อง:

- แสดง conflict
- ระบุ source ของแต่ละข้อความ
- ใช้กฎ dependency ที่ประกาศไว้ล่วงหน้า
- ลด confidence เมื่อข้อมูล stale หรือขาด
- ห้าม action ที่มีความเสี่ยงเมื่อข้อมูลไม่สดพอ
- ส่งให้เจ้าหน้าที่ตรวจและเก็บผลการ reconcile

ส่วนที่ 8: สิ่งที่ทำไปแล้วในระบบ

8.1 การเปลี่ยนแปลงล่าสุด

- เปลี่ยนหน้า One TrustX หลักเป็น Baggage Delay Control View
- เพิ่มสถานะ Check-in, Screening, Sorting, Loading, Transfer และ Arrival
- เพิ่ม overall status, bottleneck, evidence, impact, owner, deadline และ human approval
- เพิ่ม data-scope cards ว่าข้อมูลมาจากฝ่ายใด
- ระบุ N/A สำหรับจำนวนที่ต้องใช้ protected feed เช่น critical transfer-bag count
- ปรับ README และ pitch outline ให้ positioning ไม่ทับ BRS/AODB/A-CDM
- คง Event Explorer, Shared State, Conflict, Intelligence, Coordination, Scenario, Performance และ Audit เดิม
- เพิ่ม API payload baggage_control ใน shared operational state
- เพิ่ม automated tests ตรวจ stage, bottleneck, human approval และ unknown-data handling

8.2 สิ่งที่ตรวจสอบแล้ว

- Frontend production build ผ่าน
- Frontend automated tests ผ่าน 2 tests
- Backend automated tests ผ่าน 26 tests
- Baggage-specific API tests ผ่าน
- ไม่มีการลบ queue/passenger modules เดิม

8.3 สิ่งที่ยังไม่ได้ทำ

- ยังไม่เชื่อม BRS/AODB/BHS หรือ Ground Handler API จริง
- ยังไม่ validate root-cause rules กับ baggage-domain expert
- ยังไม่มีข้อมูลเที่ยวบิน กระเป๋า คน หรืออุปกรณ์จริงของ BKK
- ยังไม่มี production identity, safety case หรือ operational SLA
- ยังไม่มีผลพิสูจน์ว่าลด delayed bags หรือ coordination time ในสนามบินจริง
- ยังไม่ควบคุมอุปกรณ์หรือส่งคำสั่งไปยังสนามบิน

ส่วนที่ 9: ความเป็นไปได้และแผนพัฒนา

9.1 เหตุผลที่แนวทางนี้เป็นไปได้

- ใช้ระบบและ event ที่มีอยู่แล้วเป็นฐาน ไม่ต้องติด sensor ใหม่ทั้งสนามบินเพื่อเริ่ม PoC
- เริ่มด้วย aggregate/pseudonymous data จึงลดความเสี่ยงด้าน privacy
- แยก connector ออกจาก decision layer ทำให้ระบบต้นทางยังทำงานได้แม้ One TrustX หยุด
- เริ่มใน shadow mode: ระบบแนะนำแต่ไม่สั่งการจริง
- วัดผลด้วย workflow KPI ก่อนผลทางการเงิน

9.2 Roadmap

ระยะ	งาน	ผลลัพธ์
Phase 0: Discovery	ยืนยัน workflow, owner, cutoff และเหตุของ delay กับผู้เชี่ยวชาญ	validated process map และ data dictionary
Phase 1: Controlled prototype	scenario ปกติ/คนไม่พอ/BHS failure/transfer risk/weather	หน้าจอและ decision workflow ที่ทดสอบซ้ำได้
Phase 2: Expert validation	tabletop exercise กับ operator	feedback, false alarm, action usefulness
Phase 3: Shadow mode	feed anonymised/aggregate แบบ read-only	วัด lead time และ coordination กับข้อมูลจริง
Phase 4: Limited pilot	human-approved recommendations	operational KPI ภายใต้ขอบเขตและ governance
Phase 5: Expansion	turnaround, gate, passenger, cargo	ขยายเฉพาะ use case ที่พิสูจน์คุณค่าแล้ว

9.3 KPI ที่ควรวัด

KPI	ความหมาย	วิธีวัดในอนาคต
Detection lead time	เห็น backlog ก่อน cutoff เร็วแค่ไหน	เวลา event แรกถึง alert เทียบ baseline
Bottleneck identification time	ใช้เวลาหาสาเหตุนานเท่าไร	tabletop/shadow-mode timestamp
Coordination time	จากตรวจพบถึงมี action owner	audit timestamps
Critical bags exposed	จำนวนกระเป๋าที่เข้า risk window	authorised connection/bag feed
Recovery time	เวลาที่ backlog กลับต่ำกว่า threshold	throughput timeline
Recommendation acceptance	operator เห็นว่าคำแนะนำใช้ได้หรือไม่	accept/reject/override พร้อมเหตุผล
False alert rate	แจ้งปัญหาแต่ operation ปกติบ่อยแค่ไหน	independent operational outcome

KPI	ความหมาย	วิธีวัดในอนาคต
Bags/flights protected	ผลลัพธ์สุดท้ายหลัง intervention	matched before/after or controlled pilot

ไม่ควรใช้เปอร์เซ็นต์จาก synthetic simulation เป็น headline benefit ของสนามบินจริง

ส่วนที่ 10: ความเสี่ยง ความปลอดภัย และธรรมาภิบาล

10.1 ความเสี่ยงหลัก

- ข้อมูลจากแต่ละฝ่ายอาจมาช้า หน่วยไม่ตรง หรือมี timestamp ต่างกัน
- root cause อาจเป็นเพียง correlation ถ้าไม่มี evidence เพียงพอ
- recommendation อาจจัดลำดับผิดเมื่อข้อมูล resource ไม่ครบ
- operator อาจพึ่งระบบมากเกินไปหาก UI แสดง confidence เกินจริง
- การเชื่อมหลายองค์กรเพิ่มความเสี่ยงด้านสิทธิ์และ privacy
- ระบบกลางอาจกลายเป็นจุดล้มเหลวใหม่หากออกแบบ dependency ไม่ดี

10.2 มาตรการควบคุม

- source timestamp, ingest timestamp, schema version และ lineage ทุก event
- freshness SLA และ stale-data marker
- idempotency, duplicate detection และ sequence validation
- role-based access และ least privilege
- encryption in transit/at rest
- human approval สำหรับ operational action
- immutable/traceable audit records
- fail-safe: ระบบต้นทางทำงานต่อได้หาก One TrustX ล่ม
- shadow mode ก่อน pilot
- independent outcome label สำหรับวัด accuracy และ benefit

ส่วนที่ 11: สิ่งที่ใช้ Pitch และสิ่งที่ห้ามอ้าง

11.1 ข้อความที่ใช้ได้

One TrustX ช่วยทีม baggage และ ground operations รวมสถานะกระเป๋า เที่ยวบิน และทรัพยากรเป็น recovery view เดียว เพื่อเห็นคอขวด ประเมินผลกระทบ และจัดลำดับ action ที่เจ้าหน้าที่ต้องอนุมัติ

เราต่อยอดจาก BRS, BHS และ AODB ที่มีอยู่ ไม่ได้สร้างระบบติดตามกระเป๋าหรือฐานข้อมูลเที่ยวบินใหม่

Prototype ปัจจุบันใช้ controlled synthetic scenario เพื่อพิสูจน์ workflow, data contract, auditability และ human decision loop

11.2 ข้อความที่ห้ามอ้างในรอบนี้

- ใช้ข้อมูล operational จริงของ SKY หรือ BKK แล้ว
- ลด delayed baggage ได้จริงเป็นเปอร์เซ็นต์ใดเปอร์เซ็นต์หนึ่ง
- AI accuracy 100%
- Risk detection accuracy 86% โดยไม่อธิบายว่าเป็น proxy จำลอง
- Data consistency 100% ในสนามบินจริง

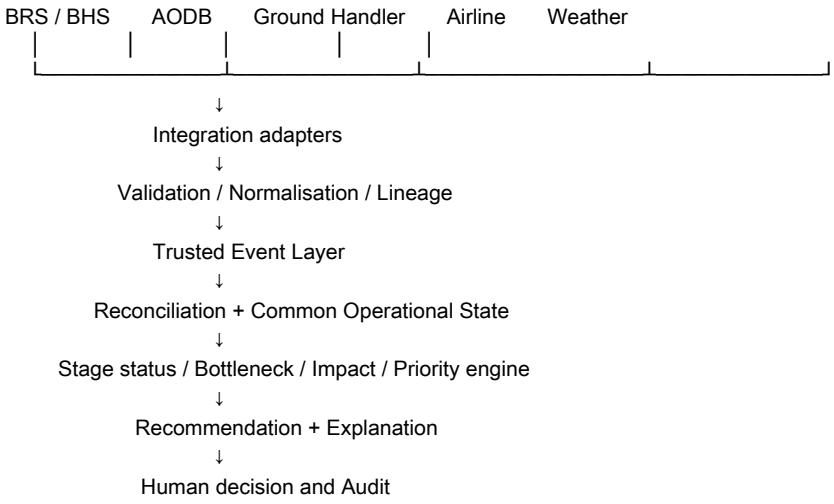
- สร้างมูลค่า 117.39 พันล้านบาท
- กระเป๋า 33.4 ล้านใบ “หาย” ทั้งหมด
- BRS ไม่มี alert หรือไม่มี analytics
- One TrustX แก้เหตุการณ์ delayed baggage ที่เกิดขึ้นจริงแล้ว

11.3 โครง Pitch 7 นาที

เวลา	เนื้อหา
0:00–1:00	ปัญหาเดียว: delayed outbound baggage และ handoff fragmentation
1:00–1:30	ผู้ใช้หลักและผู้ได้รับประโยชน์
1:30–2:30	Baggage Delay Control View
2:30–3:45	Data journey และวิธีทำงาน
3:45–4:30	แตกต่างจาก BRS/AODB/A-CDM
4:30–5:15	Feasibility และข้อมูลที่ต้องขอ
5:15–6:00	Expected KPI และวิธีวัด
6:00–7:00	Prototype plan และ future expansion

ส่วนที่ 12: Technical Appendix

12.1 Architecture ภาพรวม



12.2 Technology stack

Layer	Technology	บทบาท
Frontend	React, TypeScript, Vite	operator UI, scenario, audit และ data boundary
API	FastAPI, Python	REST interface, validation และ orchestration
Decision logic	deterministic rules, simulation, NetworkX บางโมดูล	state, scenario, comparison และ routing
Persistence	PostgreSQL, SQLAlchemy, Alembic; SQLite local preview	event, state, decision และ audit
Evaluation	Python และ optional scikit-learn	baseline/model comparison แบบ offline
Deployment	Docker Compose/local runner; frontend deployment config	reproducible demo environment

12.3 Core domain objects

Object	ความหมาย
TrustOrganization	หน่วยงานเจ้าของ event

Object	ความหมาย
TrustActor	ผู้ใช้หรือบทบาทที่ดำเนินการ
TrustScenario	นิยามสถานการณ์และ parameter
TrustEvent	เหตุการณ์พร้อม source, timestamp, checksum และ lineage
TrustConflict	กลุ่ม event ที่ขัดแย้งกัน
CommonOperationalState	สถานะร่วมที่ derive จาก evidence
TrustRecommendation	action, owner, deadline, dependency และ confidence
TrustDecision	accept/reject/override พร้อมเหตุผล
GroundTruth	label สำหรับ controlled test ไม่ใช่ production truth
ExperimentRun/KpiResult	ผลการทดลองและ KPI ที่ติด provenance

12.4 API ที่เกี่ยวข้อง

Endpoint	หน้าที่
GET /api/trustx/flights/{flight_id}/shared-state	สถานะร่วมและ baggage_control
GET /api/trustx/events	event stream ตาม scenario
GET /api/trustx/conflicts	conflict ที่ต้องตรวจ
POST /api/trustx/events/reconcile	บันทึกผล reconciliation
GET /api/trustx/predictions	risk prediction ของ controlled fixture
GET /api/trustx/recommendations	action recommendations
POST /api/trustx/recommendations/{id}/approve	human approval
POST /api/trustx/recommendations/{id}/reject	human rejection
POST /api/trustx/scenarios/{id}/run	รันเงื่อนไขทดลอง
GET /api/trustx/kpis	KPI พร้อม definition และ limitation

12.5 baggage_control contract ปัจจุบัน

```
baggage_control
  overall_status
  stages[]
    id, label, status, source
  bottleneck
    stage, reason, evidence[]
  impact
    flight, predicted_readiness_delay_min,
    affected_flights, critical_transfer_bags, note
  next_action
    action, owner, deadline, approval
  data_scope[]
    owner, data
  privacy
  source_type
```

critical_transfer_bags เป็น null ใน fixture ปัจจุบัน เพราะไม่มี authorised airline connection feed การใช้ null เป็นส่วนหนึ่งของ design เพื่อป้องกัน fabricated precision

12.6 Event validation และ reconciliation

แต่ละ event ควรมีอย่างน้อย:

- event ID และ entity ID
- event type/state
- source system และ organisation
- source timestamp และ ingest timestamp

- payload checksum
- lineage/reference
- verification status และ rule result
- conflict status และ trust status
- source type

กฎตัวอย่างใน fixture คือ baggage completion เป็น dependency ของ departure readiness ดังนั้น event BOARDING_COMPLETE ไม่สามารถทำให้เที่ยวบินเป็น ready หาก BAGGAGE_LOADING ยังอยู่ที่ 82%

12.7 Scenario และ validation pack

repository มี scenario เดิม S1–S8 และ disaster/compound S9–S14 รวมถึง flood+baggage backlog, geopolitical disruption, fire, health screening, BHS/trust-pipeline degradation และ power disruption ชุด validation สร้างแบบ deterministic หลาย seed เพื่อทดสอบ reproducibility และ workflow direction

อย่างไรก็ตาม parameter และ improvement ratio หลายค่าเป็น ASSUMPTION/SYNTHETIC ใช้ตรวจ code path ไม่ใช่หลักฐาน operational performance ตัวเลข 61%, 78%, 74%, 50%, 85.7% หรือ 0.91 ต้องไม่ถูกนำไปเรียกเป็นผลสนามบินจริง

12.8 Data quality contract สำหรับ production

ก่อนรับ event จากระบบจริง connector ต้องตรวจ:

1. schema และ units
2. airport/terminal/station identity
3. source และ event timestamp
4. freshness และ latency
5. sequence/version
6. duplicate/idempotency key
7. missing/contradictory event
8. timezone
9. lineage และ quality flag
10. authorisation และ purpose

12.9 การประเมินที่ถูกต้องเมื่อได้ข้อมูลจริง

- แยก train/validation/test ตามเวลา ไม่ random shuffle
- เปรียบเทียบกับ baseline เช่น current SOP, persistence หรือ rule baseline
- ใช้ independent outcome ไม่ใช้ label ที่ generator สร้างเอง
- รายงาน precision, recall, false alarm, lead time และ calibration
- วัด operator response และ override reason
- แยก technical performance ออกจาก operational benefit
- เริ่ม read-only/shadow mode
- ใช้ matched episode หรือ controlled rollout ก่อนสรุป causal impact

12.10 Non-functional requirements ก่อน production

- High availability และ graceful degradation
- Observability: logs, metrics, traces และ feed health

- Access control, SSO/MFA และ separation of duties
- Encryption และ key management
- Backup, recovery และ audit retention
- PDPA/data-sharing agreement
- Change management และ rule/model versioning
- Safety and human-factors review
- Incident response และ rollback
- SLA ระหว่าง data owner และ operator

12.11 Repository map

Path	เนื้อหา
frontend/	หน้าจอ One TrustX, queue simulation และ passenger modules
backend/app/trust_service.py	seed, shared state, baggage-control payload และ experiment logic
backend/app/trust_api.py	One TrustX REST endpoints
backend/app/trust_models.py	database models
data/synthetic/	fixture สำหรับ demo
data/scenarios/	scenario library และ provenance
scripts/experiments/	validation-pack generator
docs/preliminary-pitch-th.md	โครง pitch 7 นาทีล่าสุด
docs/onetrustx-pitch.md	narrative Baggage Disruption Recovery
docs/data-sources.md	source/provenance matrix
docs/validation-data-contract.md	schema และข้อจำกัดของ validation
docs/security.md	security boundaries
docs/decisions/	เหตุผลการตัดสินใจและการเปลี่ยน scope

12.12 โครงสร้างโปรเจกต์แบบละเอียด

```
onetrustx/
├── README.md                  ภาพรวม วิธีรัน ขอบเขตและข้อจำกัด
├── Makefile                   คำสั่งมาตรฐานสำหรับ boot/test/down
├── run-local.sh               เปิด local preview แบบไม่ใช้ Docker
├── schema.sql                 schema อ้างอิงระดับโครงการ
├── infra/
│   └── docker-compose.yml       PostgreSQL + API + frontend stack
├── frontend/
│   ├── src/
│   │   ├── App.tsx              routing และ application shell
│   │   ├── components/
│   │   │   ├── TrustXPages.tsx   Baggage Control, state, event, conflict,
│   │   │   │                     intelligence, coordination, audit
│   │   │   ├── Landing.tsx       หน้าแนะนำระบบ
│   │   │   ├── GuidedDemo.tsx    presenter-controlled demo
│   │   │   ├── DecisionWorkspace.tsx
│   │   │   ├── AirportMap.tsx
│   │   │   ├── DataSources.tsx
│   │   │   └── PilotPanels.tsx   pilot/evaluation panels
│   │   ├── services/api.ts      client สำหรับเรียก backend
│   │   ├── i18n.tsx             Thai/English helper
│   │   ├── locales/th.json      คำแปลภาษาไทย
│   │   ├── trustx.css           style ของ One TrustX/Baggage view
│   │   └── executive.css        style ของ executive aviation UI
│   ├── e2e/                   Playwright browser tests
│   ├── package.json             frontend dependencies/scripts
│   └── vite.config.ts           build และ local proxy
```

```
├── backend/
│   ├── app/
│   │   ├── main.py              FastAPI หลักของ queue simulation/demo
│   │   ├── engine.py            queue/simulation/routing logic เดิม
│   │   ├── trust_api.py         REST API ของ One TrustX
│   │   ├── trust_service.py     seed, shared state, baggage control,
│   │   │                        experiment และ KPI service
│   │   ├── trust_models.py      ORM ของ event/conflict/state/decision
│   │   ├── models.py            ORM ของ queue simulation/demo
│   │   ├── db.py                database session และ connection
│   │   ├── seed.py              seed data หลัก
│   │   ├── public_api.py        API สำหรับข้อมูล public context
│   │   ├── public_context.py    aggregate/transform public data
│   │   ├── public_models.py     ORM ของ public datasets
│   │   └── ingest_public.py     import normalized data
│   ├── migrations/              database schema migration
│   ├── tests/
│   │   ├── test_trustx.py       baggage/shared-state/API tests
│   │   ├── test_engine.py       simulation invariants
│   │   ├── test_api.py          endpoint behaviour
│   │   └── test_pilot.py        pilot workflow tests
│   ├── Dockerfile
│   ├── pyproject.toml
│   └── uv.lock
├── data/
│   ├── raw/                     raw public responses/cache
│   ├── normalized/              standardized public tables
│   ├── synthetic/               demo flights, baggage, ground, gate,
│   │                            passenger, predictions, recommendations
│   ├── scenarios/
│   │   └── scenario_library.json  scenario, causal chain และ provenance
│   ├── cache/                   validation runs/summary/manifest
│   ├── seeds/                   deterministic starting fixtures
│   └── dataset_manifest.json    track/type ของแต่ละ dataset
├── scripts/
│   ├── data/                    fetch, normalize, quality-report scripts
│   ├── experiments/             validation-pack/experiment runners
│   ├── generate_synthetic.py    สร้าง synthetic operational fixtures
│   ├── seed_database.py         โหลด fixture เข้า database
│   ├── render_data_pdf.py       สร้าง PDF ภาษาไทยจาก Markdown
│   ├── run_demo.py              demo automation
│   └── verify_live.py           live-stack verification
├── ml/
│   └── evaluation/evaluate.py   offline baseline/model evaluation
└── docs/
    ├── data-provenance-th.md/pdf  Project Bible ฉบับนี้
    ├── preliminary-pitch-th.md    deck structure 7 นาที
    ├── onetrustx-pitch.md         baggage narrative
    ├── architecture.md            technical architecture
    ├── api.md                     API documentation
    ├── data-dictionary.md         field/table definitions
    ├── data-sources.md            source and provenance matrix
    ├── data-quality-report.md     coverage/missingness/checksum
    ├── validation-data-contract.md
    ├── security.md                security/privacy boundary
    ├── deployment.md              environments/integration roadmap
    ├── evaluation.md              evaluation results and caveats
    └── decisions/                 Architecture Decision Records
```

ไฟล์ .env, database local, cache, build output และ virtual environment เป็น runtime/development artifact ไม่ใช่ source of truth ของข้อมูลหรือผลลัพธ์

12.13 ความสัมพันธ์ระหว่างส่วนประกอบ

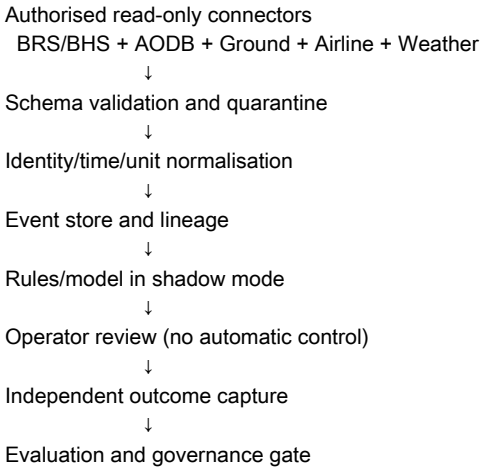
จาก	ไป	สิ่งที่ส่ง
Source systems/fixtures	Backend adapters	event และ context พร้อม timestamp/source

จาก	ไป	สิ่งที่ส่ง
Adapters	Trusted Event Layer	normalized event, lineage, checksum, quality flag
Trusted Event Layer	Reconciliation	ordered evidence และ conflicts
Reconciliation	Common Operational State	readiness, blocking dependency, confidence
Shared State	Baggage Control	stage, bottleneck, evidence และ impact
Recommendation service	Frontend	action, owner, deadline, dependency
Frontend/operator	Decision API	approve/reject และเหตุผล
Decision API	Database/Audit	immutable decision history และ outcome reference
Scenario/evaluation scripts	Cache/docs	synthetic runs, summary, manifest และ limitations

12.14 Runtime flow เมื่อเปิดหน้ากระเป๋า

1. Browser เปิด /trustx
2. Frontend เรียก shared state, events, conflicts, KPIs, scenarios, predictions และ recommendations
3. FastAPI เปิด database session และอ่าน controlled scenario S5/flight-1
4. trust_service.shared_state() คืน state เดิมพร้อม baggage_control
5. Frontend แสดง stage status ก่อน แล้วค่อยแสดง evidence/impact/action
6. ถ้าผู้ใช้เปิด Conflict Center ระบบแสดง event ที่ขัดแย้งพร้อม source
7. ถ้าผู้ใช้ reconcile ระบบบันทึก rule result และ actor
8. ถ้าผู้ใช้ approve recommendation ระบบสร้าง TrustDecision
9. การเรียก approve ซ้ำให้ผลเดิมเพื่อป้องกัน duplicate decision
10. Audit view แสดงวงจรตั้งแต่ event ถึง decision

12.15 เส้นทางข้อมูลสำหรับ production ในอนาคต



Production connector ต้องไม่เขียนกลับไปยังระบบต้นทางใน Phase แรก และถ้า One TrustX unavailable ระบบต้นทางต้องปฏิบัติงานต่อได้ตามเดิม

12.16 การทดสอบและการส่งมอบ

ระดับ	สิ่งที่ตรวจ	เครื่องมือ/ไฟล์
Unit/API	queue balance, scenario, shared state, recommendation idempotency	backend/tests/ + pytest
Frontend	routing/rendering และ UI behaviour	Vitest
Browser	presenter flow และ user experience	Playwright frontend/e2e/
Data	schema, provenance, missingness, checksum	data scripts และ quality report

ระดับ	สิ่งที่ตรวจ	เครื่องมือ/ไฟล์
Reproducibility	same scenario/seed ให้ผลเดิม	experiment scripts/manifests
Build	TypeScript และ production bundle	npm run build
Stack	API/database/frontend connectivity	Docker Compose/verify scripts

ผลตรวจล่าสุดหลังเพิ่ม Baggage Delay Control View: frontend build ผ่าน, frontend tests 2/2 และ backend tests 26/26 ผลนี้ยืนยันว่า code path ที่ทดสอบไม่แตก แต่ไม่ใช่การรับรองผลปฏิบัติการสนามบิน

ส่วนที่ 13: Q&A สำหรับกรรมการ

ต่างจาก BRS อย่างไร

BRS เป็นระบบสำคัญสำหรับ tracking/reconciliation และมี alert บางประเภทอยู่แล้ว One TrustX ไม่ติดตามซ้ำ แต่เชื่อม baggage signal กับ flight deadline, ground resource, connection และ disruption เพื่อสร้าง recovery priority ข้ามฝ่าย ความแตกต่างนี้ต้องยืนยันกับ capability จริงของ SKY ในช่วง discovery

ต่างจาก A-CDM อย่างไร

A-CDM ครอบคลุม collaboration และ milestone ระดับสนามบิน/เที่ยวบิน One TrustX รอบนี้เป็น use case เฉพาะด้าน baggage disruption และไม่อ้างว่าสร้าง A-CDM ใหม่

เป็นเพียง dashboard หรือไม่

ถ้ามีเพียงการแสดง KPI ก็จะเป็น dashboard ความตั้งใจของระบบคือมี event validation, conflict reconciliation, dependency rules, impact calculation, prioritised recommendation, human decision และ audit แต่คุณค่าของ logic เหล่านี้ยังต้องพิสูจน์กับ expert และข้อมูลจริง

AI อยู่ตรงไหน

Prototype ปัจจุบันใช้ deterministic/rule-based logic เป็นหลักเพื่อให้ตรวจสอบย้อนกลับได้ ML/AI เป็น candidate สำหรับ anomaly, risk และ prioritisation ในอนาคต แต่จะไม่ promote จนกว่าจะชนะ baseline และผ่าน validation ด้วยข้อมูลจริง

ถ้าไม่มีข้อมูล Ground Handler จะทำอย่างไร

รอบ preliminary ใช้ synthetic scenario และระบุช่องว่างตรงไปตรงมา Phase แรกต้องทำ data discovery กับ Ground Handler หากไม่มี resource feed ระบบยังแสดง baggage milestone ได้ แต่ไม่ควรสรุป root cause ว่าเกิดจากคนหรืออุปกรณ์

ทำไมไม่เริ่มจาก passenger tracking

เพราะ SKY มี Smart Flows และ biometric/passenger systems อยู่แล้ว ขณะที่ feedback รอบนี้ต้องการปัญหาหนึ่งเรื่องที่ชัด กระเป๋ามี milestone และ handoff ที่กำหนดได้ จึงเหมาะกับการพิสูจน์ coordination architecture มากกว่า

ระบบตัดสินใจแทนคนหรือไม่

ไม่ ระบบแนะนำพร้อม evidence, confidence และ limitation ผู้มีอำนาจเป็นผู้อนุมัติ ปฏิเสธ หรือแก้ไข และทุกการตัดสินใจถูก audit

จะรู้ได้อย่างไรว่าระบบมีประโยชน์จริง

ต้องทดลองกับ operator และข้อมูลจริงใน shadow mode วัด detection lead time, bottleneck identification, coordination time, false alert, recovery time และ bags/flights protected เทียบกับ baseline ก่อนสรุป benefit

---

ส่วนที่ 14: แหล่งอ้างอิงหลัก

1. SKY ICT — Baggage Reconciliation System
2. SKY ICT — Airport Operational Database
3. IATA — Baggage Tracking and Resolution 753
4. SITA — Baggage IT Insights 2026

5 Airports of Thailand — Baggage Status Information, 26–29 September 2026

6 Open-Meteo documentation

7 OpenSky REST API

8 OurAirports open data

9 CAAT

10 เอกสารภายใน repository: data-sources.md, data-quality-report.md, validation-data-contract.md, security.md, architecture.md, preliminary-pitch-th.md และ ADR ใน docs/decisions/

ข้อสรุปสุดท้าย

One TrustX ในทิศทางล่าสุดไม่ใช่แนวคิด “รวมทุกข้อมูลแล้วให้ AI ตัดสินใจทุกอย่าง”
แต่เป็นระบบช่วยกู้สถานการณ์ที่เริ่มจากปัญหาเดียว มีผู้ใช้ชัด ข้อมูลชัด ขอบเขตชัด และสามารถตรวจสอบเหตุผลได้:

เริ่มจาก delayed baggage ใช้ระบบที่ SKY มีอยู่แล้ว เชื่อมเฉพาะสัญญาณที่จำเป็น แสดงคอขวดและผลกระทบ จัดลำดับ action ให้เจ้าหน้าที่ตัดสินใจ และพิสูจน์คุณค่าด้วย shadow-mode evidence ก่อนขยายไป use case อื่น

สถานะปัจจุบันคือ prototype สำหรับพิสูจน์แนวคิดและ workflow ไม่ใช่ระบบ production หรือหลักฐานผลลัพธ์ของสนามบินจริง ความน่าเชื่อถือของโครงการจึงมาจากการแยกข้อเท็จจริง ข้อมูลจำลอง สมมติฐาน และผลคำนวณออกจากกันอย่างโปร่งใส